The Network Simplex Algorithm

Ari Holcombe Pomerance, Baihan Liu, and Grace Truong
University of Colorado Denver

Minimum Cost Flow Problem Formulation

Definition: Given a digraph G with capacities $u: E(G) \to \mathbb{R}_+$ and numbers b: $V(G) \to \mathbb{R}$ such that $\sum_{v \in V(G)} b(v) = 0$. A **b-flow** in (G, u) is a function f: $E(G) \to \mathbb{R}_+$ with $f(e) \le u(e)$ for all $e \in E(G)$ and

$$\sum_{e \in \delta^+(v)} f(e) - \sum_{e \in \delta^-(v)} f(e) = b(v)$$

for all $v \in V(G)$.

Formulation
Instance: A digraph G with capacities $u: E(G) \to \mathbb{R}_+$, numbers c: $E(G) \to \mathbb{R}$ indicating the cost of each edge, and numbers b: $V(G) \to \mathbb{R}$ such that $\sum_{v \in V(G)} b(v) = 0$ (can have multiple source and sink nodes).

Task: Find a b-flow f whose cost $c(f) := \sum_{e \in E(G)} f(e)c(e)$ is minimum (or decide that none exists)

Theorem 9.2
Let G be a digraph with capacities u and numbers b (defined the same as above). There exists a b-flow if and only if

$$\sum_{e \in \delta^+(X)} u(e) \ge \sum_{v \in X} b(v) \text{ for all } X \subseteq V(G).$$

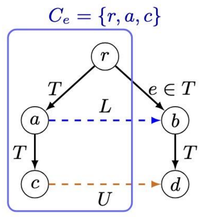
We can find one or decide that none exists in $O(n^2\sqrt{m})$ time, where $n: |V(G)|$ and $m := |E(G)|$.

Important Definitions

Given a min-cost flow problem (G, u, b, c):

Spanning Tree Structure.

- quadruple: (r, T, L, U)
- r : root
- $E(G) = T \sqcup L \sqcup U$
- $(V(G), T)$ is a spanning tree

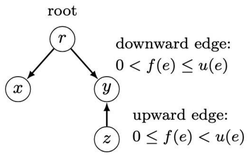


Associated b-flow.

Non-tree edge: $f(e) = 0$ for $e \in L$, $\quad f(e) = u(e)$ for $e \in U$.
Tree edges are determined by the cut formula:

$$f(e) = \sum_{x \in C_e} b(x) + \sum_{a \in U \cap \delta^-(C_e)} u(a) - \sum_{a \in U \cap \delta^+(C_e)} u(a).$$

Strong Feasibility.



Potential π and Reduced Cost c_π.

Choose the potential π with
$\pi(r) = 0, \quad c_\pi(e) = 0$ for all $e \in T$.
For $e = (v, w)$,
$c_\pi(e) = c(e) + \pi(v) - \pi(w)$.
For $e \notin T$, $c_\pi(e)$ is the unit cost of the corresponding circuit augmentation.

Relation to LP

LP	Network Simplex
basis	spanning tree structure (r,T,L,U)
basic variable	tree edges T
nonbasic variable	edges in L,U
dual variable	potentials π
reduced cost	$c_\pi(e)$
pivot direction	augment along C
ratio test	bottleneck δ

Network Simplex Algorithm

Input: strongly feasible (r, T, L, U) | Output: optimum b-flow.

1. Compute the **b-flow f** and the **potential π** associated with (r, T, L, U).

2. Choose an **entering edge**
   $e \in L$ with $c_\pi(e) < 0$, or
   $e \in U$ with $c_\pi(e) > 0$.
   If no such edge exists, stop.

3. Form the **fundamental circuit C**.

4. Compute the maximum feasible augmentation $\delta := \min_{e' \in E(C)} u_f(e')$.
   Choose the **leaving edge e_0** by the last-blocking-edge rule.

5. Pivot the tree by setting $T := (T \cup \{e\}) \setminus \{e_0\}$, and update L, U.

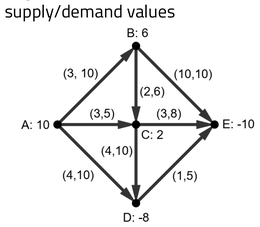
6. **Augment** f by δ along C. Update node potentials.

7. Repeat from Step 2.

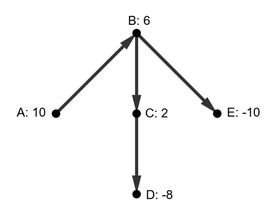
Key idea:
Augment around a fundamental circuit.
Pivot to a new strongly feasible spanning tree structure.

Algorithm Example

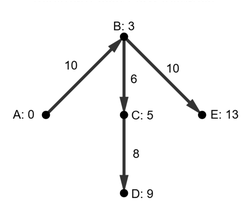
Input: Note for the Minimum Cost Flow instance the values on the edges are (costs, capacities) and the values on the vertices are the supply/demand values



Minimum Cost Flow Instance



Strongly Feasible Spanning Tree Structure

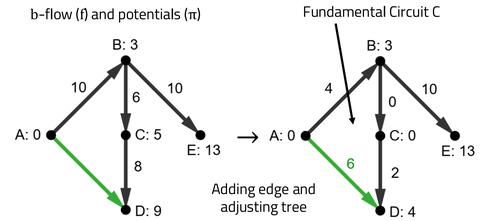


b-flow (f) and potentials (π)

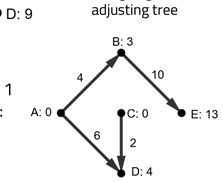
$c_\pi(e_{AC}) = -2$
$c_\pi(e_{AD}) = -5$
$c_\pi(e_{CE}) = -5$
$c_\pi(e_{DE}) = -3$

Let $e = e_{AD}$

Fundamental Circuit C



Tree after 1 iteration:



Theorems

Proposition 9.22
Any instance of the minimum cost flow problem either has a **spanning tree solution** as an optimal solution, or it has **no optimal solution**.

Proof Idea:
- Start with an optimal solution f, and consider the **undirected cycle** $C = (V(G), \{e \in E(G) : 0 < f(e) < u(e)\})$.
- Split C into two **directed cycles** C' and C''.
- **Augment** by the minimum capacity along C' and C'' to get optimal solutions f' and f''.
- At least one of these has fewer edges e with $0 < f(e) < u(e)$ than f, and so we have an optimal **spanning tree solution**.

Corollary 9.23
The **basic solutions** of

$$\{x \in \mathbb{R}^{E(G)} : 0 \le x_e \le u(e) \quad (e \in E(G))$$
$$\sum_{e \in \delta^+(v)} x_e - \sum_{e \in \delta^-(v)} x_e = b(v) \quad (v \in V(G))\}$$

are precisely the **spanning tree solutions** of the minimum cost flow problem.

Proposition 9.22 and Corollary 9.23 together allow us to formulate the minimum cost flow problem as a linear programming problem. Spanning tree solutions have three types of edges:

1. Edges with zero flow
2. Edges at capacity
3. **Edges with positive flow but under capacity**

The edges in this third category extend to form a (directed) spanning tree of our network.

Proposition 9.25 (Complexity)
Given an instance of the minimum cost flow problem and a spanning tree structure, the b-flow f can be computed in $O(m)$ time, and the potential π can be computed in $O(n)$ time.

Proposition 9.26
If the potential π for a feasible spanning tree structure satisfies
- $c_\pi(e) \ge 0$ for all $e \in L$; and
- $c_\pi(e) \le 0$ for all $e \in U$,

then that feasible spanning tree structure is associated with an optimum b-flow.

Theorem 9.27
The Network Simplex Algorithm stops after finitely many iterations and returns an optimum solution.

Proof Idea:
- The updated b-flow and potential at the end of one iteration can still be associated with (r, T, L, U). We are keeping a basic solution each time.
- Based on how we are choosing δ (or the edge that is leaving the tree) we maintain strong feasibility.
- From Prop 9.26, the computed flow the algorithm calculates is optimum.
- No two iterations results in the same (f, π)
  - Each iteration, cost is reduced by $|c_\pi(e)|\delta$.
  - If $\delta > 0$ then total cost is reduced → can't repeat a state
  - If $\delta = 0$ (degeneracy) then cost remains the same and
    - If $e = e_0$: number of edges that could enter the tree would decrease → can't repeat a state
    - If $e \ne e_0$: potential increases → can't repeat a state

Code / Example

NetworkX
Network Analysis in Python

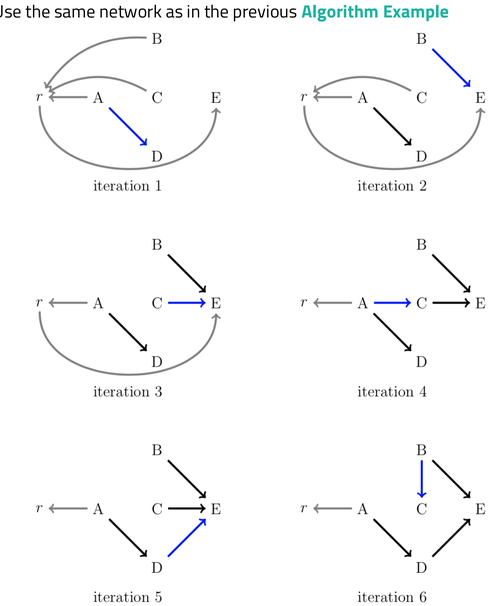
Implementation: `network_simplex` function in NetworkX

Input: A network G with capacities u, weights c, and supply/demands b

Output:
- `flowCost`: the **minimum cost** of a b-flow
- `flowDict`: the **values of b-flow** along each edge for an optimal solution of the minimum cost flow problem

The function performs a version of the Network Simplex Algorithm on the provided directed graph.

Example:
Use the same network as in the previous **Algorithm Example**



- Starts by creating a **dummy node** r and **edges** (v, r) with capacity $u(v, r) = \infty$ for all $v \in V(G)$. This is the root of the spanning tree.
- This serves as a **trivial starting solution** (an initial strongly feasible spanning tree).
- At each pivot step, we insert the **entering edge** and remove the **leaving edge** from our network.
- Continue until there is no entering edge → **Done!**

Bibliography

B. Korte, J. Vygen, Combinatorial Optimization, 6th edition, Theory and Algorithms, Springer, 2018

NetworkX Documentation. Network Simplex function. https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.flow.network_simplex.html